Formal verification of distributed, data-intensive microservice applications using Petri Nets

Shubham**
SAP Labs
Bangalore, India
Computer Science
IIIT
Bangalore, India
shubham@iiitb.ac.in

Meenakshi D'Souza
IIIT
Bangalore, India
meenakshi@iiitb.ac.in

Abstract

Modern enterprise applications are built using distributed, data-intensive microservice architectures and lack formal verification frameworks for proving the correctness of these applications. We propose a new verification approach that integrates enterprise-grade applications with object-centric process mining. In our approach, event logs from our applications are used to mine object-centric Petri nets. The mined Petri nets are then analyzed using the GreatSPN tool [2] to verify liveness and safety properties. We conducted experiments on simulated event data to demonstrate that high-quality models can be automatically mined and used for formal verification. Our framework enables scalable and automated analysis of complex microservice systems, advancing formal methods in data-intensive distributed architectures.

CCS Concepts

• **Software and its engineering → Software verification**; *Distributed systems organizing principles*; *Petri nets*; *Process validation.*

Keywords

Formal Verification, Microservice, Petri Nets

ACM Reference Format:
Shubham and Meenakshi D'Souza. 2026. Formal verification of distributed, data-intensive microservice applications using Petri Nets. In *Innovations in Software Engineering Conference (ISEC 2026), February 19–21, 2026, Jaipur, India.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3796563.3796583

1 Introduction

The shift from monolithic to microservice architectures has revolutionized enterprise software development, enabling greater scalability, maintainability, and technological diversity. However, this architectural evolution has introduced significant verification challenges: while individual microservices can be tested in isolation, the emergent behavior of their interactions often leads to subtle bugs, deadlocks, and correctness violations that are difficult to detect through traditional testing approaches.

Our research addresses this gap by proposing a novel framework that integrates Object-Centric process mining [16] with formal verification techniques. By leveraging the rich event data naturally produced by microservice interactions, we can automatically mine formal models and verify critical correctness properties without requiring manual specification or central orchestration. Our framework has been prototyped and its usage on publicly available logs illustrate its applicability for verification.

The considered problem is defined in Section 2 which is followed by Section 3 where a brief background on the various methodologies are given. An example involving real-world transactions on which our verification framework is applied is given in Section 4. The framework is presented in Section 5 highlighting the limitations of current approaches. Section 6 presents details on how the framework evaluates and makes the verification effective. Related work followed by future work concludes the paper.

2 Problem Statement

There is a critical gap in the formal verification capabilities of modern enterprise applications built using distributed, data-intensive microservice architectures. In contrast to monolithic applications, microservices operate on separate timelines and use diverse mechanisms, such as APIs, message brokers, and shared databases for communication.

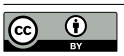
Core Research Question: Can object-centric process mining be leveraged to formally verify and analyze the correctness properties of existing distributed, data-intensive microservice applications?

Distributed microservice applications pose several challenges, such as fragmented business logic across independent codebases, obscure data flow due to layered API interactions, hidden failures such as corruption and deadlocks due to complex dependencies, limited formal verification due to missing process models, and scalability constraints due to high communication volumes.

3 Technical Background and Preliminaries

Object-Centric Event Logs (OCEL): OpenXES is specifically designed to handle case-centric event logs where events are associated with a single process instance. In contrast, microservice architectures involve interactive entities (users, orders, products) participating across multiple overlapping process instances. OCEL 2.0 [4] captures this non-linear process interaction by associating each event with multiple objects of different types.

*Author's tuition fee is sponsored by SAP Labs India.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ISEC 2026, Jaipur, India
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2503-6/26/02
https://doi.org/10.1145/3796563.3796583

Object-Centric Petri Nets: OCPNs [16] are specialized abstractions of colored Petri nets designed for automated discovery from OCEL logs. OCPNs focus on object types and cardinality constraints and abstract away complex guards and arc inscriptions. Thick solid arrows represent variable arcs and allow transitions to consume or produce an arbitrary number of tokens, while non-variable arcs work with a single token.

1-Safe Petri Nets: A 1-safe net is a subclass of Place/Transition nets where no place holds more than one token in any reachable marking. 1-safe nets guarantee a finite state space [9], which is crucial for exhaustive analysis. 1-safe nets do not preserve multiplicity but allow decidable verification.

CTL Model Checking: Computational Tree Logic provides branching-time temporal logic for specifying safety and liveness properties. CTL formulas use path quantifiers (A = all paths, E = some path) with temporal operators (F = eventually, G = always). Properties expressed as $en(T)$ (transition T enabled) or $\#PlaceName$ (token count) enable human-readable verification of microservice behaviors.

4 Motivating Example

We demonstrate our framework using a procure-to-pay application [15] that captures the P2P process from simulated real-world SAP transactions.

Our framework (Figure 1) processes OCEL 2.0 event logs through four integrated stages, mining Object-Centric Petri Nets and unfolding them into 1-safe nets for specific object types.

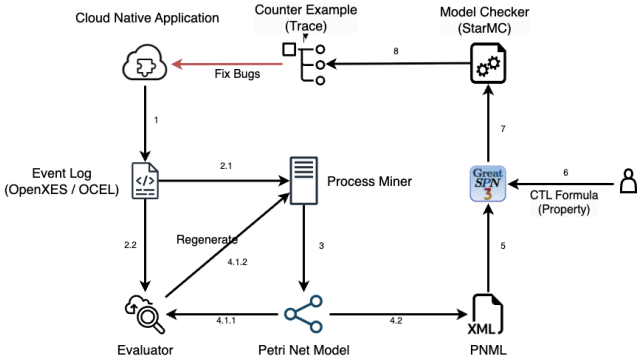


Figure 1: Framework Overview: (1) Event log generation, (2) Object-Centric Petri Net discovery, (3) Unfolding to 1-safe nets, (4) CTL model checking.

Figure 2 shows a partial Object-Centric Petri Net for the procure-to-pay process, including the complete invoice receipt flow. The thick solid arrows represent *variable arcs*, which allow transitions to consume or produce arbitrary numbers of tokens of the same type when they fire, capturing the multi-object nature of microservice interactions.

We unfold the discovered Object-Centric Petri Nets into 1-safe Petri nets for each object type. Figure 3 shows the resulting unfolded Petri net imported into GreatSPN for verification analysis.

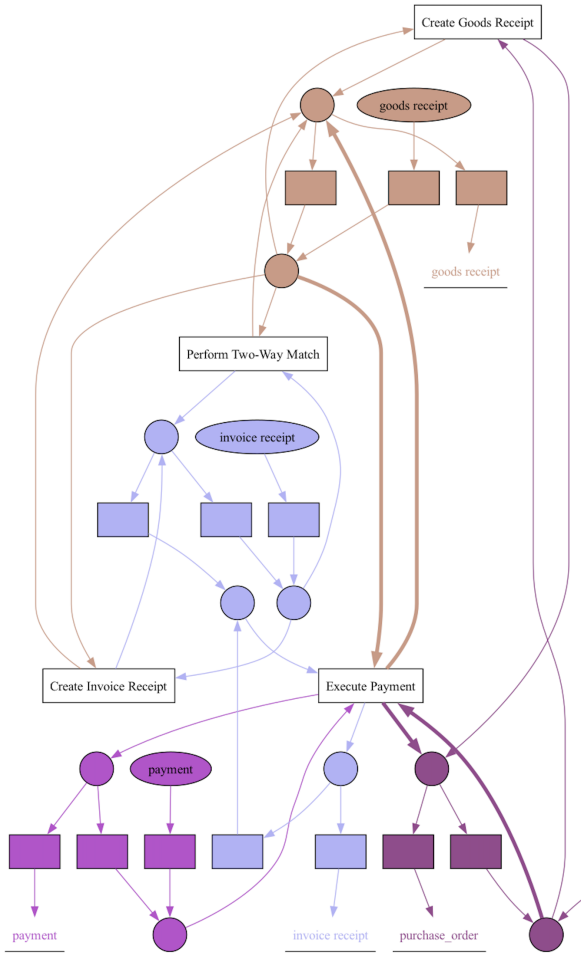


Figure 2: Invoice Receipt Processing Object-Centric Petri Net mined from Procure-to-Pay dataset.

We formalize the correctness properties in CTL for verification. In our CTL formulations, $en(T)$ denotes that transition T is enabled at a given marking (Petri net state) - it evaluates to true if transition T can fire and false otherwise. The duplicate payment property ensures payment occurs only once per invoice: $AG(en(Execute_Payment) \rightarrow AG(!en(Execute_Payment)))$.

Our evaluation generates counterexamples demonstrating that place *p_9* violates the formula when tokens reach it multiple times through *skip_9* transitions, confirming the known bug in the public logs.

5 Potential Solutions

5.1 Current Approaches and Limitations

Existing approaches include metric temporal logic [13], orchestration based methods [8], CSP [14], and colored Petri Nets [11], but suffer from scalability issues or undecidable analysis.

Event Log Format Evolution: Traditional XES event logs assume case-centric views where events are associated with single process instances. However, microservice architectures involve

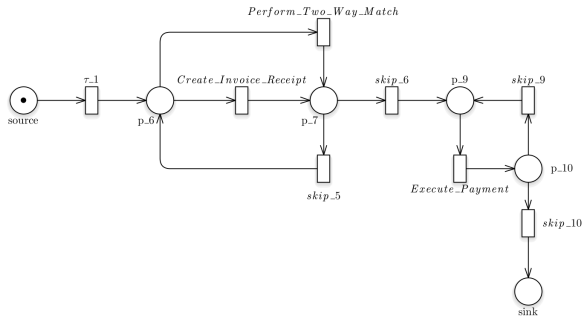


Figure 3: Unfolded 1-safe net for Invoice Receipt Petri Net imported into GreatSPN.

interactive entities (users, orders, products) participating in multiple overlapping processes. OCEL 2.0 [4] addresses this limitation by associating each event with multiple objects of different types, capturing the full network of process interactions beyond linear case-event models.

OCEL to Object-Centric Petri Net Mapping: The OCEL format directly supports Object-Centric Petri Net discovery. Each object type in OCEL becomes a place type in the OCPN, while events become transitions that can interact with multiple object types simultaneously.

1-Safe Net Abstraction: We lose multiplicity while making the verification problem decidable by unfolding OCPNs into 1-safe nets with a focus on specific object types. Object instances become tokens, and the resulting net captures the object flow for one of the processes. This abstraction is essential to retain the behavioral properties of different processes in the original system while enabling exhaustive state space analysis for formal verification.

5.2 Our Proposed Framework

Our framework provides automated, scalable analysis of microservice applications through four integrated components:

Event Log Generation: We use message queue dumps and generic plugins added as dependencies to intercept and log events in the OCEL 2.0 [4] format. Our plugin implementation is non-invasive and platform-agnostic, relying only on the communication between microservices to generate event logs.

Petri Net Discovery: We identify objects and generate Object-Centric Petri nets from OCEL logs using PM4Py's discovery algorithms [16] while preserving multi-object interactions.

Unfolding to 1-Safe Nets: We transform Object-Centric Petri Nets into classic 1-safe Petri Nets for each object type, ensuring finite state spaces for exhaustive analysis.

CTL Model Checking: We formalize correctness properties as CTL formulas and verify them using GreatSPN [2] with the StarMC model checker [3]. Our approach supports safety, liveness, deadlock freedom, and reachability properties, with counterexample generation for violated properties.

A prototype implementation of the framework is available at https://github.com/prophet1906/ddia_analyzer.

6 Evaluation Methodology

We evaluated our framework on multiple OCEL 2.0 datasets, including Procure-to-Pay [15]. Our evaluation methodology has three dimensions:

Log-Model Quality Assessment: The focus of this assessment was to quantify the behavior captured in event logs against the mined Petri nets. Higher *Replay fitness* [1] (0–1 scale) shows that event log behavior can be reproduced by the discovered Object-Centric Petri net. Higher *Precision* [10] limits over-generalization and evaluates the extent to which the enabled transitions in the model are linked to the activities observed in the logs. *Generalization* [6] assesses the alignment between the log and the mined Petri net to ensure robust behavior beyond the observed logs. *Simplicity* [5] measures the structural complexity of the mined Petri nets and favors compact and interpretable models.

Technical Performance Metrics: We assessed discovery time, memory usage, and scalability on a MacBook Pro M4 Pro with 48 GB RAM.

Verification Effectiveness: We evaluated our ability to detect correctness violations through counterexample generation, testing deadlock freedom using $EF\ ndeadlock$, starvation detection with $AF(\#PlaceName \neq 0)$, and custom properties specific to each dataset.

6.1 Success Metrics

Our framework demonstrates success across the following critical dimensions:

Technical Performance: We observed consistent replay fitness of 1.0 across all datasets, with discovery times ranging from 1.42 to 3.70 seconds. Memory usage remained stable and scaled predictably from 32.83 MB to 83.13 MB with varying log sizes.

Model Quality Metrics: We observed that precision varied from 0.091 to 1.0 depending on process complexity. Simpler processes (Customer Order, Payment) achieved a perfect precision of 1.0. We also observed that the generalization score consistently exceeded 0.82, indicating robust model–log alignment. The simplicity metric ranged from 0.47 to 1.0, with higher scores indicating a more streamlined process.

Verification Capabilities: We were able to successfully verify key properties such as deadlock freedom across all object types, liveness providing request completion guarantees, starvation prevention ensuring service availability, reachability ensuring proper request routing, and system emptiness detecting idle states. We were also able to successfully identify duplicate payment bugs in P2P logs with counterexample generation, demonstrating practical bug detection capabilities.

Scalability Boundaries: Our approach is capable of handling datasets with up to 35K+ events efficiently but cannot verify properties requiring token identity, timing constraints, or load-balancing fairness due to 1-safe net limitations.

7 Related Work

Existing approaches to formal verification of microservice applications include:

Logic-Based Approaches: Marconi et al. [13] proposed modeling the behavior of Apache Storm topologies using metric temporal

logic (CLTLoc with counters) for formal verification. The proposed approach demonstrated the feasibility of automated verification but suffered from scalability issues due to counter-logic undecidability and required approximations.

Orchestration-Centric Methods: Camilli et al. [8] proposed a new pipeline to connect design-time formal modeling with run-time monitoring, which enables properties verified offline to be continuously enforced during system execution. Their later work [7] proposed a formal framework to model and verify safety, liveness and structural correctness of microservice process flows at design time. However, the approach relies on a central orchestration framework, Netflix's Conductor in this case, limiting applicability to choreography-based microservice architectures.

Process Algebraic Approaches: Martin [14] modeled microservices as CSP (Communicating Sequential Processes) processes and used refinement-based model checking to formally verify various properties at design time. The approach required substantial abstraction to maintain model checking feasibility, limiting practical applicability.

Data-Aware Modeling: Aalst et al. [16] introduced the approach of directly discovering multi-object process models in the form of Object-Centric Petri Nets from logs without converting them into single-case traces. This work provided the theoretical foundation for our approach but focused on process mining rather than verification.

Specialized Applications: Recent work includes Röwekamp et al. [11] addressing the verification of correct compensation execution and recovery properties in Saga pattern using colored Petri Nets (though with undecidable analysis), and Kamboj et al. [12] demonstrating an anomaly detection framework for microservices to identify deviations from mined Petri Nets.

We combined Object-Centric process mining with formal verification to provide automated analysis and bug detection without requiring central orchestration or manual modeling of the system.

8 Future Work and Research Challenges

Quantitative Property Analysis: Our approach cannot handle latency, availability and load distribution fairness properties due to limitations of 1-safe nets. We plan to explore timed and stochastic Petri net variants to enable quantitative analysis.

Multi-Object Composition: We unfold OCPNs to object-specific Petri nets which limits our analysis when independent events across different objects influence each other. We plan to explore Petri net composition techniques with overlapping places/transitions to create unified Petri nets supporting cross-object event interactions.

Dynamic Architecture Verification: Static Petri nets cannot model auto-scaling microservices with flexible topologies. We plan to investigate alternative approaches like message passing chains for dynamic topology support.

Industrial Validation: Beyond synthetic datasets, we plan extensive validation with real enterprise applications.

References

[1] Wil van der Aalst Alessandro Berti. 2019. Reviving Token-based Replay: Increasing Speed While Improving Diagnostics. In *International Workshop on Algorithms & Theories for the Analysis of Event Data 2019*. https://ceur-ws.org/Vol-2371/ATAED2019-87-103.pdf

[2] Elvio Gilberto Amparore, Gianfranco Balbo, Marco Beccuti, Susanna Donatelli, and Giuliana Franceschinis. 2016. 30 years of GreatSPN. In *Principles of Performance and Reliability Modeling and Evaluation*. Springer, 227–254.

[3] Elvio Gilberto Amparore, Susanna Donatelli, and Francesco Gallà. 2022. starMC: An automata based CTL* model checker. *PeerJ Comput. Sci.* 8 (2022), e823. doi:10.7717/peerj-cs.823

[4] Alessandro Berti, Istvan Koren, Jan Niklas Adams, Gyunam Park, Benedikt Knopp, Nina Graves, Majid Rafiei, Lukas Liß, Leah Tacke Genannt Unterberg, Yisong Zhang, Christopher Schwanen, Marco Pegoraro, and Wil M. P. van der Aalst. 2024. OCEL (Object-Centric Event Log) 2.0 Specification. doi:10.48550/ARXIV.2403.01975

[5] Fabian Rojas Blum. 2015. Metrics in process discovery. https://api.semanticscholar.org/CorpusID:15882411

[6] J. C. A. M. Buijs, B. F. Van Dongen, and W. M. P. Van Der Aalst. 2014. Quality Dimensions in Process Discovery: The Importance of Fitness, Precision, Generalization and Simplicity. *International Journal of Cooperative Information Systems* 23, 01 (March 2014), 1440001. doi:10.1142/S0218843014400012

[7] Matteo Camilli, Carlo Bellettini, and Lorenzo Capra. 2018. Design-Time to Run-Time Verification of Microservices Based Applications. In *Software Engineering and Formal Methods*, Antonio Cerone and Marco Roveri (Eds.). Springer International Publishing, Cham, 168–173. doi:10.1007/978-3-319-74781-1_12

[8] Matteo Camilli, Carlo Bellettini, Lorenzo Capra, and Mattia Monga. 2018. A Formal Framework for Specifying and Verifying Microservices Based Process Flows. In *Software Engineering and Formal Methods*, Antonio Cerone and Marco Roveri (Eds.). Springer International Publishing, Cham, 187–202. doi:10.1007/978-3-319-74781-1_14

[9] Allan Cheng, Javier Esparza, and Jens Palsberg. 1995. Complexity results for 1-safe nets. (Aug. 1995), 117–136. doi:10.1016/0304-3975(94)00231-7

[10] David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Doug Tygar, Moshe Y. Vardi, Gerhard Weikum, Jorge Muñoz-Gama, and Josep Carmona. 2010. A Fresh Look at Precision in Process Conformance. In *Business Process Management*, Richard Hull, Jan Mendling, and Stefan Tai (Eds.). Vol. 6336. Springer Berlin Heidelberg, Berlin, Heidelberg, 211–226. doi:10.1007/978-3-642-15618-2_16

[11] Daniel Moldt Jan Henrik Röwekamp, Manuela Buchholz. 2021. Petri Net Sagas. https://ceur-ws.org/Vol-2907/paper4.pdf

[12] Priyanka Kamboj, Cyrille Artho, Roberto Guanciale, Reyhaneh Jabbarvand, and Brighten Godfrey. 2025. Leveraging Petri Nets for Workflow Anomaly Detection in Microservice Architectures. In *Application and Theory of Petri Nets and Concurrency*, Elvio Amparore and Lukasz Mikulski (Eds.). Vol. 15714. Springer Nature Switzerland, Cham, 219–241. doi:10.1007/978-3-031-94634-9_11

[13] Francesco Marconi, Marcello M. Bersani, Madalina Erascu, and Matteo Rossi. 2016. Towards the Formal Verification of Data-Intensive Applications Through Metric Temporal Logic. In *Formal Methods and Software Engineering*, Kazuhiro Ogata, Mark Lawford, and Shaoying Liu (Eds.). Springer International Publishing, Cham, 193–209. doi:10.1007/978-3-319-47846-3_13

[14] Jeremy M. R. Martin. 2021. Designing and Verifying Microservices Using CSP. In *IEEE Concurrent Processes Architectures and Embedded Systems Virtual Conference (COPA)*. IEEE, San Diego, CA, USA, 1–4. doi:10.1109/COPA51043.2021.9541471

[15] Gyunam Park and genannt Unterberg Leah Tacke. 2023. Procure-To-Payment (P2P) Object-centric Event Log in OCEL 2.0 Standard. doi:10.5281/zenodo.8412920

[16] Wil M.P. Van Der Aalst and Alessandro Berti. 2020. Discovering Object-centric Petri Nets. *Fundamenta Informaticae* 175, 1-4 (Sept. 2020), 1–40. doi:10.3233/FI-2020-1946